



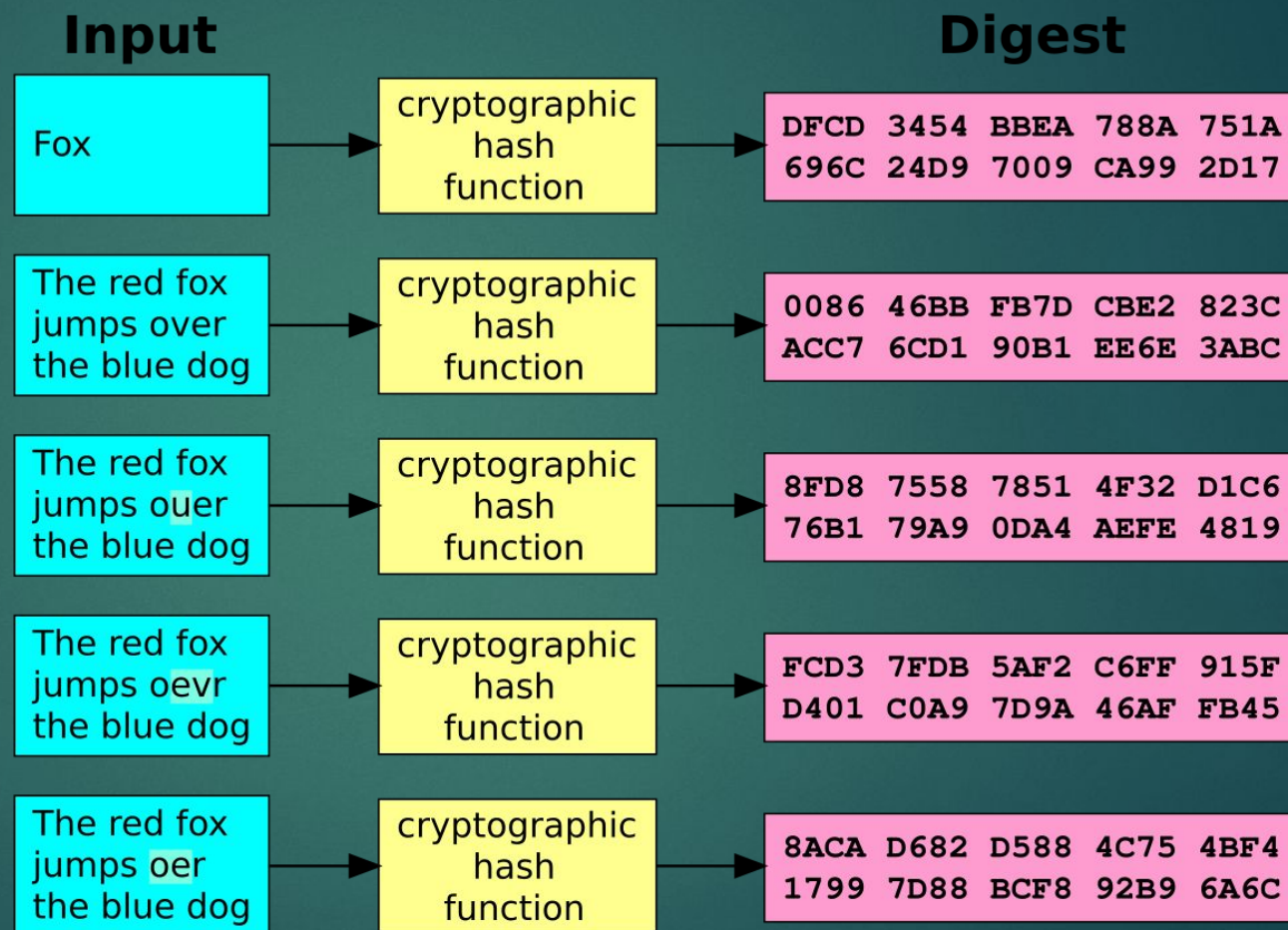
Introduction

rapide: Blockchain

RAPHAEL BISCHOF

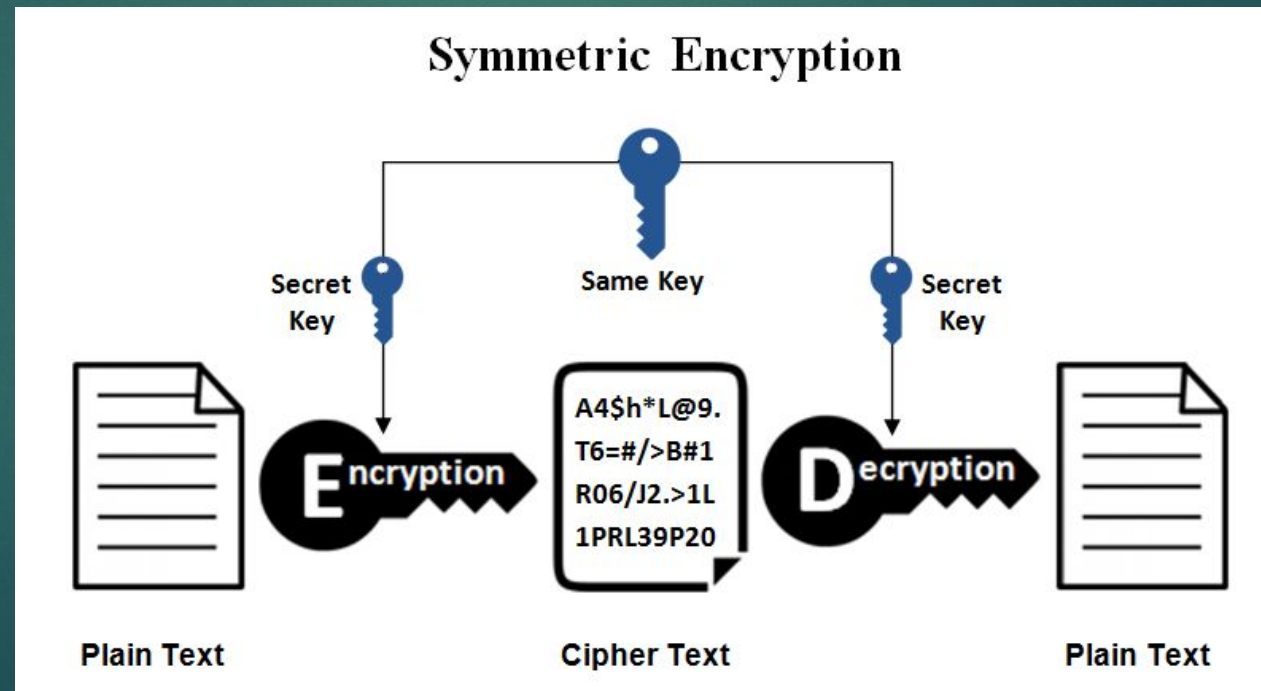
Lexique

► Hashage



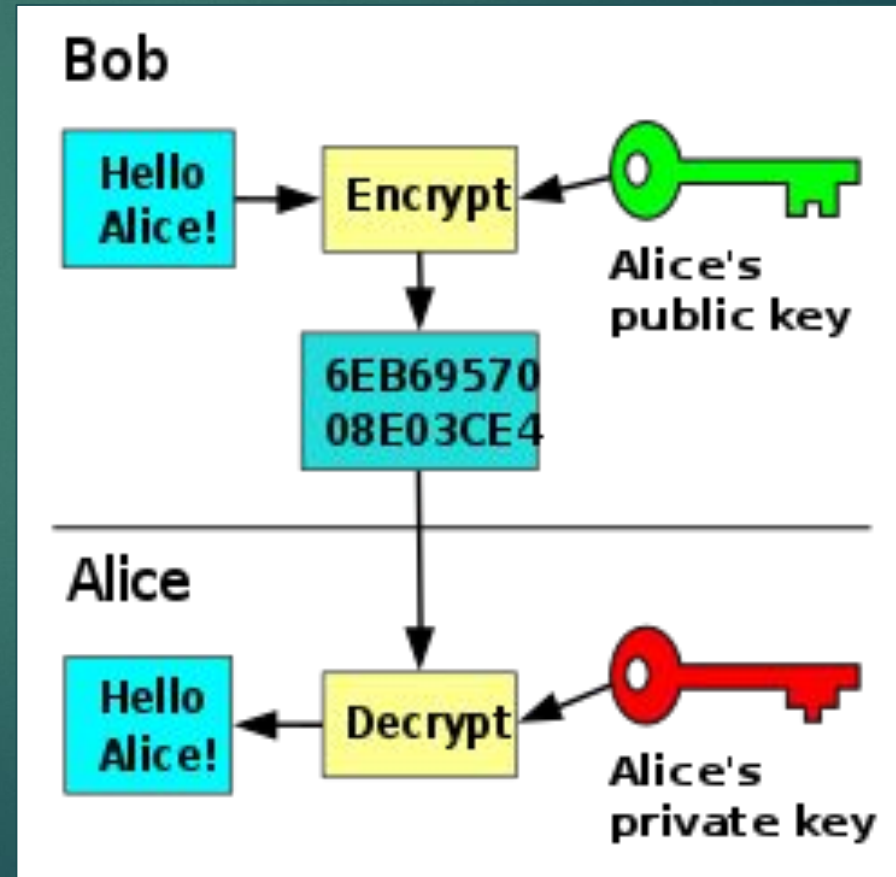
Lexique

- ▶ Symmetric encryption
 - ▶ Secret partagé



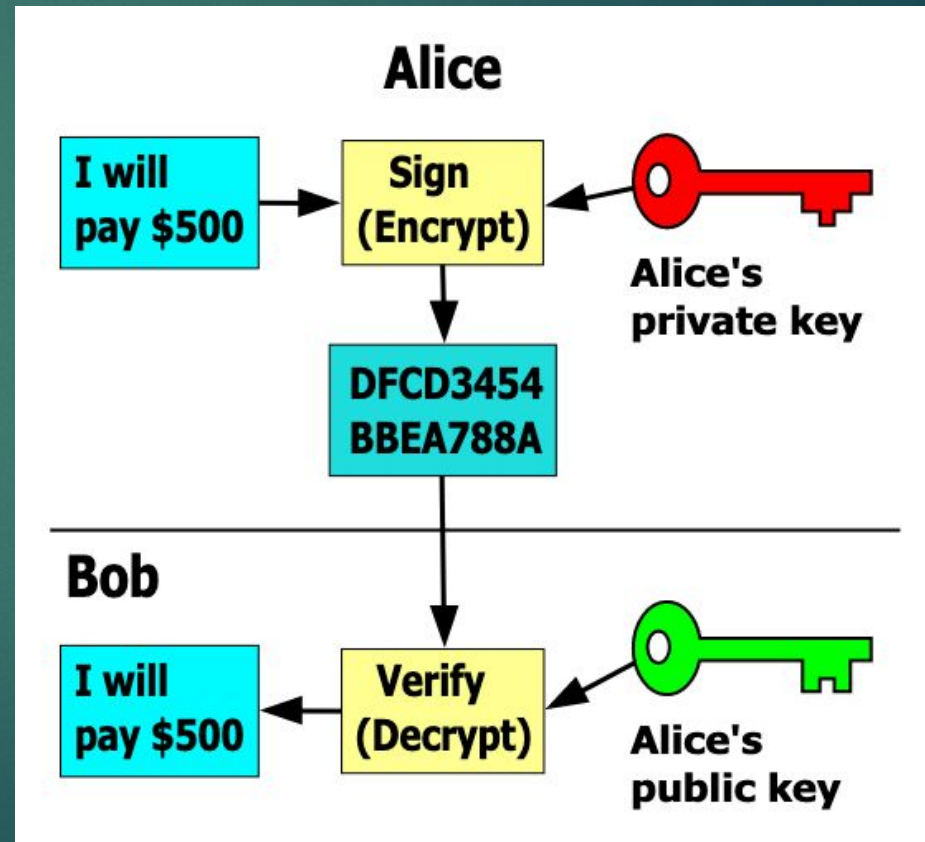
Lexique

- ▶ Asymmetric encryption
 - ▶ Secret non partagé
 - ▶ Encryption



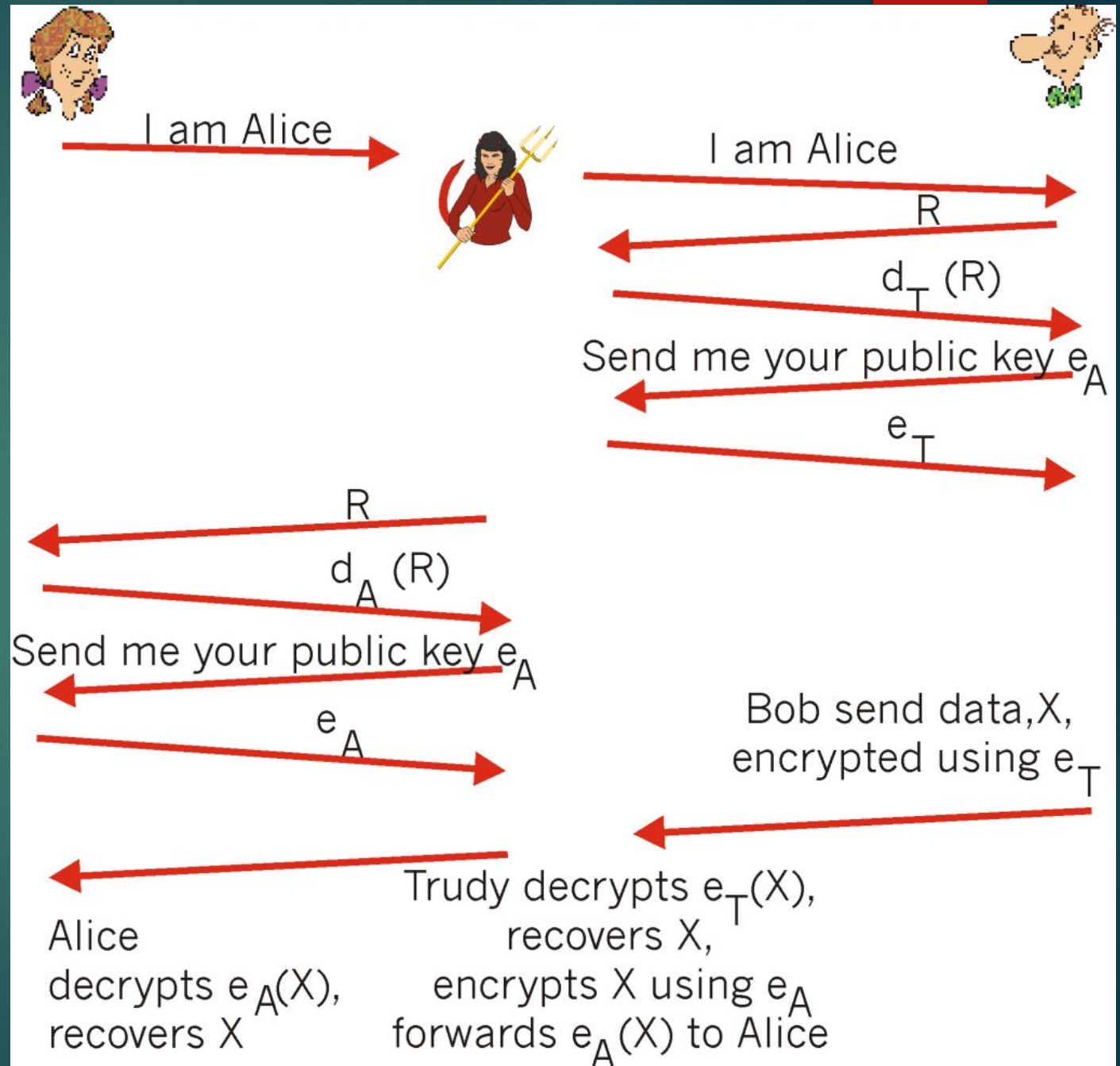
Lexique

- ▶ Asymmetric encryption
 - ▶ Secret non partagé
 - ▶ Signing



Lexique

- ▶ Attaque man in the middle



Définition

- ▶ Chaîne de blocks
 - ▶ Contenu d'un bloc:
 - ▶ Transactions signées par les envoyeurs



Lexique

- ▶ Node
 - ▶ Ordinateur/serveur synchronisant la blockchain
- ▶ Block
 - ▶ Ensemble de transaction signées
- ▶ Genesis block
 - ▶ Premier bloc d'une blockchain
- ▶ Transaction
 - ▶ Donation d'argent de la part d'une adresse bitcoin vers une autre adresse
- ▶ Ethereum: Crypto tokens
 - ▶ Ether: Monnaie/token d'ethereum
 - ▶ Gas: Unité de calcul de cout de computation
- ▶ Hash
 - ▶ Transformation d'une donnée à sens unique pouvant être transmise sans risque qu'une personne retrouve la donnée d'origine
 - ▶ Exemple: SHA 256 de test ->
9f86d081884c7d659a2feaa0c55ad015a3bf4f1b2b0b822cd15d6c15b0f00a08

Fonctionnement - Réseau - P2P

- ▶ Blockchain bitcoin
 - ▶ Découverte de nodes
 - ▶ Premiers nodes hardcodés dans le client
 - ▶ Seconde version avec les nodes récupérés d'un channel IRC
 - ▶ DNS address pour demander une liste de nodes
- ▶ Ensemble de méthodes reconnues par les clients:
 - ▶ Méthodes importantes
 - ▶ Ping
 - ▶ GetBlocks
 - ▶ GetBlockTxn
 - ▶ Tx

Genesis block

```
GetHash()      = 0x0000000000019d6689c085ae165831e934ff763ae46a2a6c172b3f1b60a8ce26f
hashMerkleRoot = 0x4a5e1e4baab89f3a32518a88c31bc87f618f76673e2cc77ab2127b7afdeda33b
txNew.vin[0].scriptSig      = 486604799 4
0x736B6E616220726F662074756F6C69616220646E6F63657320666F206B6E697262206E6F20726F6C6C65636E61684320393030322F6E614A2F333020736
56D695420656854
txNew.vout[0].nValue        = 5000000000
txNew.vout[0].scriptPubKey =
0x5F1DF16B2B704C8A578D0BBAF74D385CDE12C11EE50455F3C438EF4C3FBCF649B6DE611FEAE06279A60939E028A8D65C10B73071A6F16719274855FEB0F
D8A6704 OP CHECKSIG
block.nVersion = 1
block.nTime     = 1231006505
block.nBits     = 0x1d00ffff
block.nNonce    = 2083236893

CBlock(hash=0000000000019d6, ver=1, hashPrevBlock=00000000000000, hashMerkleRoot=4a5e1e, nTime=1231006505, nBits=1d00ffff,
nNonce=2083236893, vtx=1)
  CTransaction(hash=4a5e1e, ver=1, vin.size=1, vout.size=1, nLockTime=0)
    CTxIn(COutPoint(000000, -1), coinbase
04ffff001d0104455468652054696d65732030332f4a616e2f32303039204368616e63656c6c6f72206f6e206272696e6b206f66207365636f6e642062616
96c6f757420666f722062616e6b73)
    CTxOut(nValue=50.00000000, scriptPubKey=0x5F1DF16B2B704C8A578D0B)
  vMerkleTree: 4a5e1e
```

Génération de block - Etapes

1. Broadcast des transactions
2. Récupération par le générateur du bloc des transactions en attente
3. Insertion dans le bloc
4. Broadcast du nouveau bloc

Pour la simplicité de l'explication, les futures slides seront par rapport à la blockchain Ethereum

Génération de bloc - Transaction

Contenu d'une transaction:

- ▶ Destinataire de la transaction
- ▶ Emetteur de la transaction
- ▶ Montant de la transaction
- ▶ Gas price : montant d'ether par étapes payé au block miner
- ▶ Start gas : montant maximum de gaz alloué à la transaction
- ▶ Signature: Signée par la clef privée de l'adresse

Exemple:

Txn d'1 ether, 15 GWEI (donc 0.000000015 ether) gas price, start gas 21

Cout de la transaction:

$15 * 21 = 315 \text{ GWEI} = 0.000000315 \text{ Ether} = 0.000064 \text{ USD (05/02/2020)}$

Seulement $1 - 0.000000315 = 0.999999685$ Ether au destinataire

Génération de bloc - Creation de block

Récupération des transactions en attente (intéressantes en premier en terme de reward)

Droit de générer un bloc en fonction de la blockchain

- ▶ Proof of work
- ▶ Proof of stake

Génération de bloc - Creation de block

Exemple de proof of work

Hasher "Hello world!" avec n'importe quel suffixe avec une valeur de hash finale inférieure à 2^{240}

```
"Hello, world!0" => 1312af178c253f84028d480a6adc1e25e81caa44c749ec81976192e2ec934c64 = 2^252.253458683
"Hello, world!1" => e9afc424b79e4f6ab42d99c81156d3a17228d6e1eef4139be78e948a9332a7d8 = 2^255.868431117
"Hello, world!2" => ae37343a357a8297591625e7134cbea22f5928be8ca2a32aa475cf05fd4266b7 = 2^255.444730341
...
"Hello, world!4248" => 6e110d98b388e77e9c6f042ac6b497cec46660deef75a55ebc7cfd65cc0b965 = 2^254.782233115
"Hello, world!4249" => c004190b822f1669cac8dc37e761cb73652e7832fb814565702245cf26ebb9e6 = 2^255.585082774
"Hello, world!4250" => 0000c3af42fc31103f1fdc0151fa747ff87349a4714df7cc52ea464e12dcd4e9 = 2^239.61238653
```

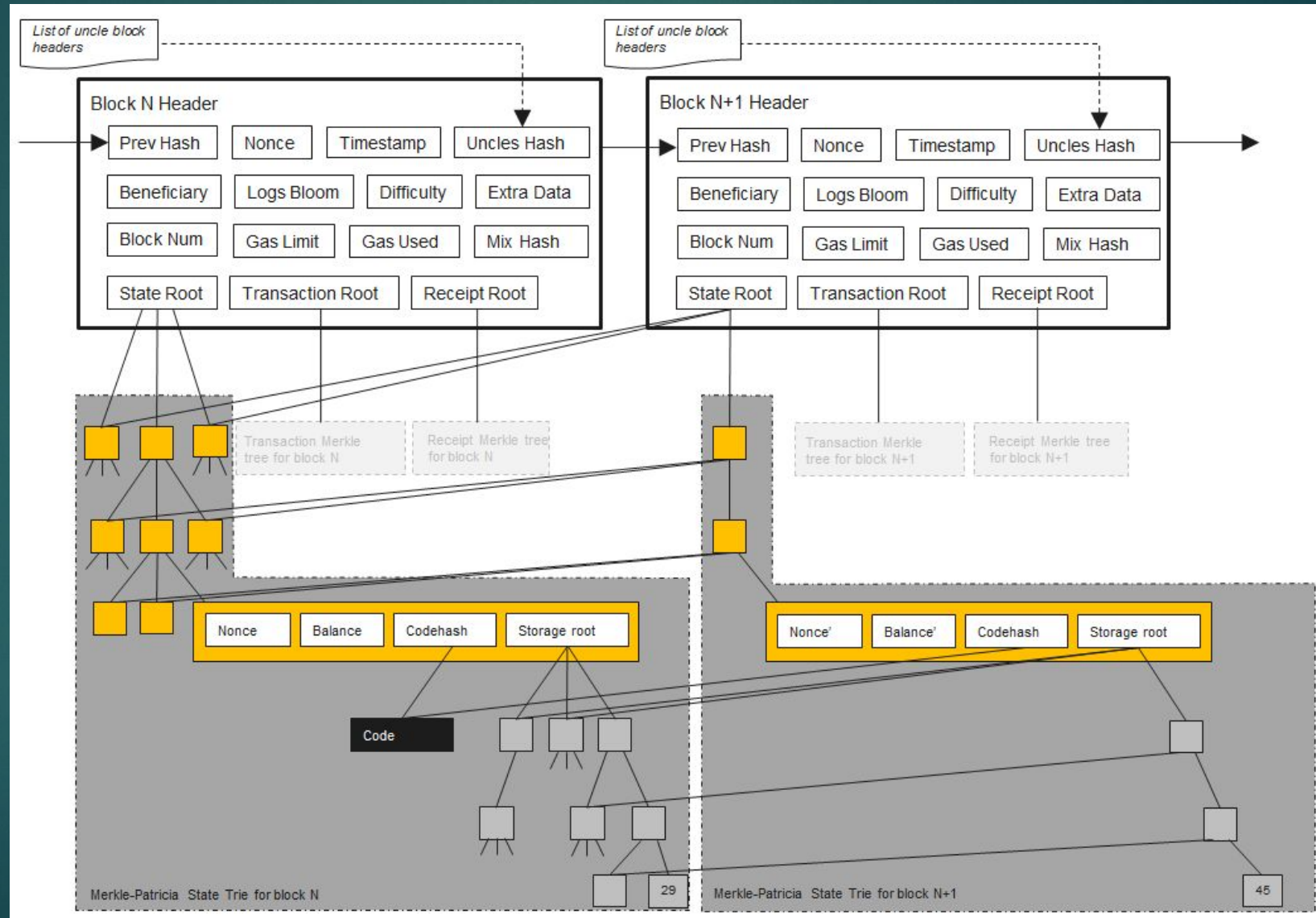
Difficulté héritée des blocs d'avant et du temps de résolution

Pas très eco-friendly, Ecosia n'est pas très content de ça

Génération de bloc - La vraie

- ▶ Insère toutes les transactions dans le bloc
- ▶ Donne sa recipient address
- ▶ Encaisse le Reward pour le bloc + les transactions fees

Génération de bloc - La vraie



Proof of stake

Proof of stake



The probability of validating a new block is determined by how large of a stake a person hold.



The validators do not receive a block reward, instead they collect network fees as their reward.



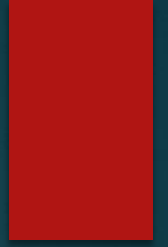
Proof of stake systems can be much more cost and energy efficient than proof of work, but are less proven.

0.08% à 0.2% réduit à pratiquement 0 pour ETH PoW à PoS

Génération de bloc - Creation de block

- ▶ Proof of stake : Aléatoire dans les plus grands possesseurs de monnaie/tokens
 - ▶ Prérequis:
 - ▶ Doit avoir les pièces depuis + de X temps
 - ▶ Ne doit pas avoir été choisi avant

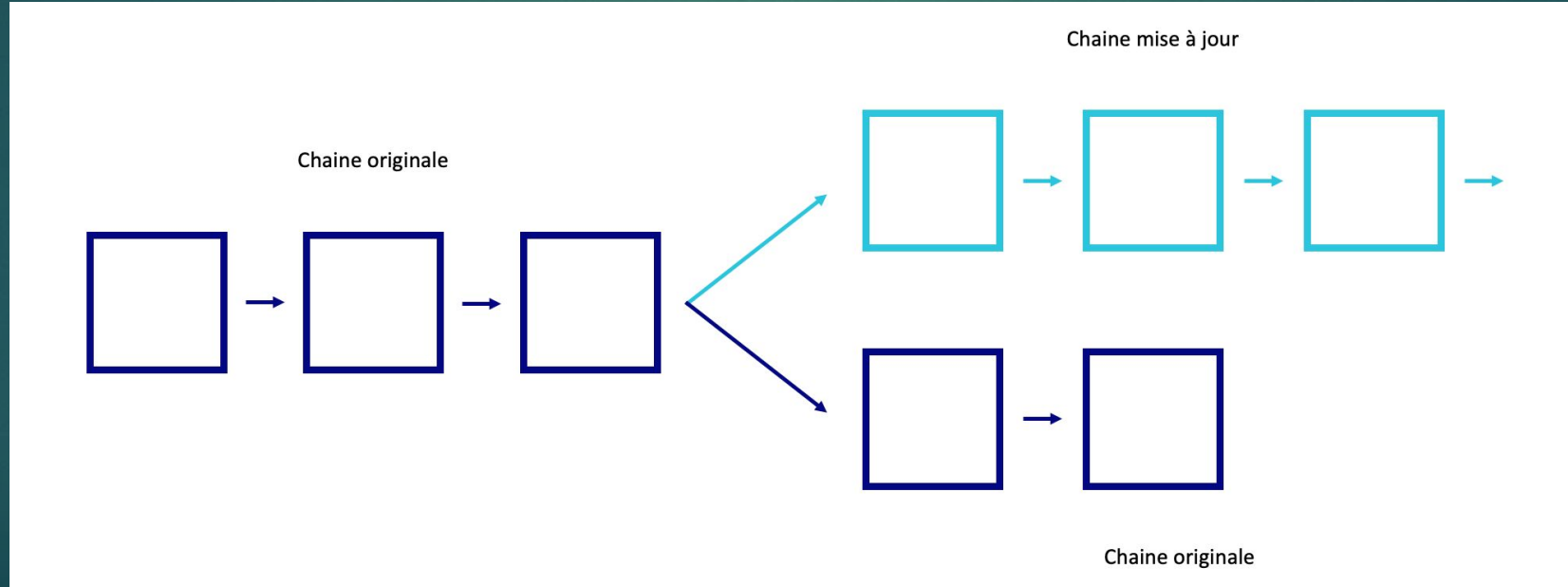
Limites blockchain (L1)



- ▶ Bitcoin
 - ▶ 1 bloc toutes les 10 minutes
 - ▶ Taille de bloc maximale 1 mo
 - ▶ < 4000 transactions dans un bloc
 - ▶ 1 transaction toute les 7 secondes
- ▶ Ethereum
 - ▶ 10-15 secondes par blocs
 - ▶ Pas de taille de bloc maxi
 - ▶ Contrainte de temps de synchronisation des transactions en P2P
 - ▶ Environ 15 transactions par secondes
- ▶ Visa
 - ▶ 1700 transactions par secondes en pic
- ▶ Mastercard
 - ▶ Jusqu'à 20000 transactions par secondes en pic

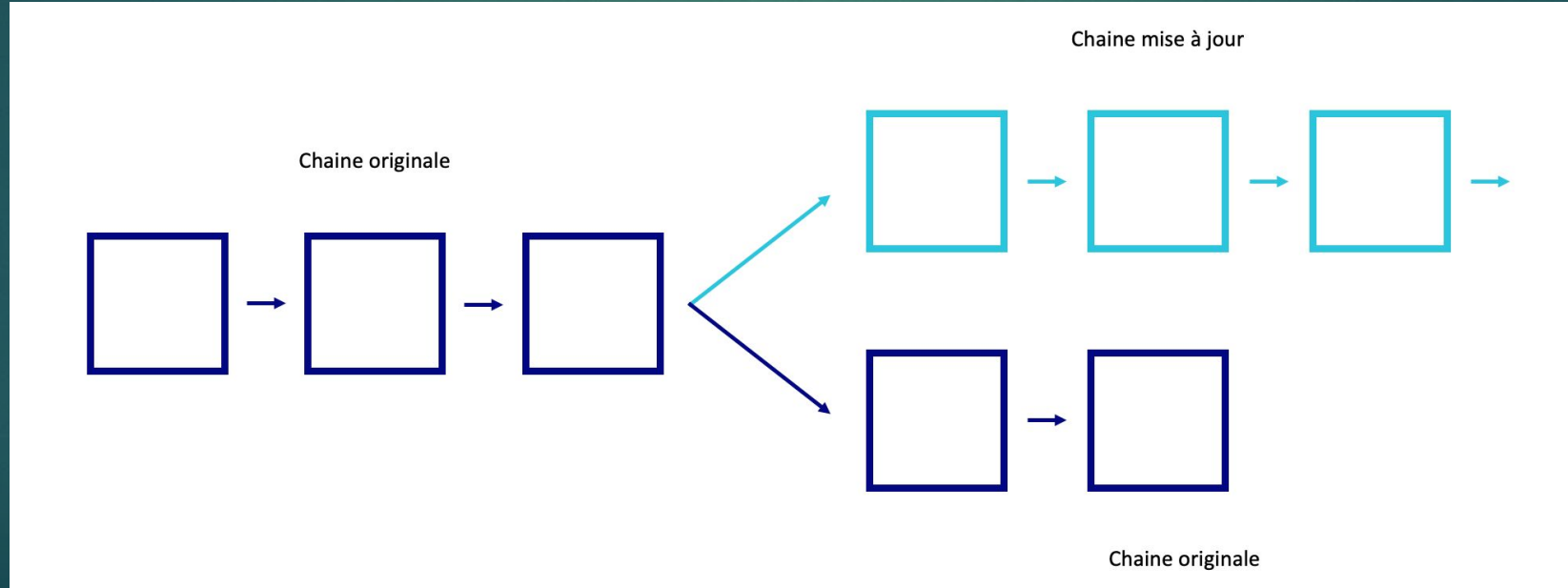
Forks et hard forks

- ▶ Le fonctionnement d'une blockchain est intrinsèquement lié au code exécuté du côté client
 - ▶ Si une mise à jour n'est pas rétrocompatible, cela resultera en un hard fork



Génération de block concurrents

- ▶ Plusieurs blocs peuvent être générés de manière concurrent (uncle blocks), ceux-ci sont ajoutés dans les blocs suivants pour les “acknowledge”. La plus grand PoW gagne. Bitcoin les appellent orphan et ne les intègre pas.



Limites blockchain

- ▶ Scalabilité
 - ▶ Nécessité de synchroniser tous les blocs précédents et de les vérifier
 - ▶ Possible de download des snapshots mais risque de snapshot corrompus

Limites blockchain

- ▶ Attaque des 50%
 - ▶ Pour une proof of work, si quelqu'un possède + de 50% de la puissance de calcul, il pourra prendre le contrôle de la blockchain en validant certaines transactions mais pas d'autres
- ▶ Replay attacks
 - ▶ Répétitions de transactions signé dans un autre fork
- ▶ Pertes de la private key

Applications

- ▶ Non applicable pour remplacer la monnaie normale actuellement
- ▶ Smart contracts applicable pour différent cas d'utilisation, exemple une “assurance” pour les paysans
 - ▶ Code du smart contract
 - ▶ Client A: Assureur
 - ▶ Client B: Paysan
 - ▶ L'assureur (A) crée un contrat qui va vérifier automatiquement les données de météo pour voir si il n'y a pas eu de conditions destructrices pour les cultures, un montant d'argent est bloqué dans le smart contract
 - ▶ Le paysan (B) paie le smart contract pour avoir accès à cette assurance
 - ▶ A l'échéance du smart contract, un node va executer le code et vérifier le service météo, si les conditions ont été mauvaises, l'argent sera reversé à B autrement l'argent est automatiquement rendu à A.

Aberration

Une blockchain “privée” est inutile, elle pourrait être remplacée par une base de données dans ce cas là.

Le remplacement de la monnaie commence à devenir technologiquement possible. La contrainte étant l'adoption et la multitude de technos différentes (plusieurs L1 qui contiennent chacun plusieurs L2)

La valeur des blockchains ne sont indexées sur rien, leur valeur est purement conceptuelle

Smart contract

- ▶ Langage non turing complet sur la blockchain bitcoin
 - ▶ Disponible sur les blockchains “tokenisés”, c'est à dire avec des jetons
 - ▶ Pas de jetons/gas sur bitcoin qui ne donne pas de protection sur les boucles infinies, du coup, pas de boucles
- ▶ Permet l'exécution de code
 - ▶ Execution payée par le gas mis à disposition
 - ▶ (Ethereum -> Solidity)

Smart contract Example

```
1
2 pragma solidity ^0.4.24; Compiler version
3
4 library SafeMath { Library
5
6     /**
7      * @dev Multiplies two numbers, reverts on overflow. Code comment
8      */
9     function mul(uint256 a, uint256 b) internal pure returns (uint256) {
10         if (a == 0) {
11             return 0;
12         }
13         uint256 c = a * b;
14         require(c / a == b);
15         return c;
16     }
17     ...
18 }
19
20 interface IERC20 { Interface declaration (subcontract)
21
22     function totalSupply() external view returns (uint256);
23     function balanceOf(address who) external view returns (uint256); Interface functions
24     ...
25 } Subcontract
26
27 contract ERC20 is IERC20 { Interface implementation
28
29     using SafeMath for uint256;
30
31     mapping (address => uint256) private _balances;
32     mapping (address => mapping (address => uint256)) private _allowed;
33     uint256 private _totalSupply;
34
35     /**
36      * @dev Total number of tokens in existence
37      */
38     function totalSupply() public view returns (uint256) {
39         return _totalSupply;
40     }
41
42     /**
43      * @dev Gets the balance of the specified address.
44      * @param owner The address to query the balance of.
45      * @return An uint256 with the amount owned by the passed address.
46      */
47     function balanceOf(address owner) public view returns (uint256) {
48         return _balances[owner];
49     }
50     ...
51 }
52
53 contract MyCoin is ERC20 { Contract inheritance
54     string public symbol;
55     string public name;
56     uint8 public decimals;
57
58     function MyCoin() public { Constructor
59         symbol = "MC";
60         name = "MyCoin";
61         decimals = 18;
62     }
63     ...
64 }
65 }
```


Smart contract NFT Example

```
RandomApe.sol M X
contracts > RandomApe.sol
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.4;
3
4 import "@openzeppelin/contracts/token/ERC721/ERC721.sol";
5 import "@openzeppelin/contracts/token/ERC721/extensions/ERC721Burnable.sol";
6 import "@openzeppelin/contracts/access/AccessControl.sol";
7 import "@openzeppelin/contracts/utils/Counters.sol";
8
9 contract RandomAPE is ERC721, ERC721Burnable, AccessControl {
10     using Counters for Counters.Counter;
11
12     bytes32 public constant MINTER_ROLE = keccak256("MINTER_ROLE");
13     Counters.Counter private _tokenIdCounter;
14
15     constructor() ERC721("Random APE", "RA") {
16         _grantRole(DEFAULT_ADMIN_ROLE, msg.sender);
17         _grantRole(MINTER_ROLE, msg.sender);
18     }
19
20     function _baseURI() internal pure override returns (string memory) {
21         return "ipfs://ipfs_link/";
22     }
23
24     function safeMint(address to) public onlyRole(MINTER_ROLE) {
25         uint256 tokenId = _tokenIdCounter.current();
26         _tokenIdCounter.increment();
27         _safeMint(to, tokenId);
28     }
29
30     // The following functions are overrides required by Solidity.
31
32     function supportsInterface(bytes4 interfaceId)
33         public
34         view
35         override(ERC721, AccessControl)
36         returns (bool)
37     {
38         return super.supportsInterface(interfaceId);
39     }
40 }
```